Reviving a Dead Game — How Straw Hat Samurai: Duels Came Back to Life

Build-along course: how the game was decompiled, byte-patched and protocol-emulated back to life.

Ch 1 — What died, exactly

Before reviving anything, diagnose the death. "The game is down" hides several separate failures.

On 2026-10-30 we probed every address the game could possibly need. Four different objects, four different verdicts:

Thing	Status	Verdict
strawhatsamuraiduels.com	DNS NXDOMAIN	dead — domain not even registered anymore
r.playerio.com (connect API)	HTTP 500	dead — server exists but its games are gone
cdn.playerio.com GameFS (assets)	HTTP 200	still alive in 2026! Assets were mirrored before it also dies
whipflash.com/api.php (payments)	HTTP 502	dead but irrelevant — payments aren't on the boot path

And one more death the table can't show: **Flash Player itself** (Adobe killed the plugin Dec 2020). That one is solved separately by an emulator (Ch 9).

Key insight: "The game is dead" was actually three dead services, one dead platform (Flash Player), plus one surprise survivor (the CDN) — and one of the "dead" services (payments) turns out not to matter on the boot path. Diagnosis first = you fix only what's broken instead of despairing at a vague problem.

Ch 2 — What a .swf actually is

SWF ("Small Web Format") = a whole game in ONE file: compiled ActionScript 3 bytecode, vector art, bitmaps, fonts, sound, animation data, even embedded data blobs.

The SHS Duels web client (`shs3_payload_2016-11-30.swf` , 6.9 MB, preserved byte-identical from a 2016 web capture) carried 27 embedded data blobs — maps, items, quests — plus all game logic. Two facts shaped everything:

- It was wrapped in a commercial protector ("SWFProtection 2.x"), which encrypts bytecode chunks — a wall we had to get past.
- We also had a second, unwrapped copy. The Android build (v0.682) shipped differently, and its decompiled output was cleaner. Web v0.717 + Android v0.682 together gave us near-complete readable logic.

Rule: NEVER bet on one file. Two builds of the same game = two independent vantage points; one often fills the holes the other hides.

Ch 3 — Front end vs back end

The single most important mental model: a "dead game" almost always means only HALF the system is dead.

- **Front end** = code running on YOUR machine. The SWF: rendering, input, combat rules, animations, even the "connecting…" UI. You own a free copy — it's immortal.
- **Back end** = code running on servers. Accounts, saves, gold transactions, matchmaking, admin data (MOTD, maintenance flags). Rented — it dies the day the rent stops.

The front end is a question-asker: it sends "here are my credentials, log me in", "save this character", "what's my gold?" — and it carries hardcoded logic for READING the answers. It knows the answer format in advance; there's just nobody answering anymore.

The whole trick in one sentence: we never restored the back end. We wrote a NEW one that answers questions in the dialect the front end already understands.

What died vs what survived:

- Died: real account logins, saved characters on the DB, gold as a real currency, live multiplayer opponents, purchase processing.
- Survived in the SWF: all combat logic, quests, NPCs, shops, the overworld map & skills, all art/sound /music, the UI including the parsing code for server replies — even the save-file format (so we can save locally).

Ch 4 — Concept: reverse engineering

Not hacking, not guessing — a disciplined method for learning how a machine works from the outside.

- Forward engineering: idea design code product.
- Reverse engineering: product observation working model of how it must have been built.

Three activities, and we did all of them:

1. **Observe behaviour.** Run the SWF in an emulator, click "Play as Guest", watch the network tab. We saw it fire requests at `http://www.strawhatsamuraiduels.com/server/…` — and fail. Observation turns "dead" into a list of specific requests.
2. **Extract structure.** Open the binary, pull out embedded art/sounds AND compiled code. That's where the decompiler lives (Ch 6–7).
3. **Reconstruct intent.** The client contains every question AND every answer-parser. From those two, the shape of the missing answers is deducible with zero server access.

Beginner trap: never guess what the server said. Every response format must be traceable to parsing code that reads it. If you can't cite the line of client code that consumes a field, you don't know the field exists.

Ch 5 — Concept: decompilation — microscope, not time machine

Computers never run the developer's source. A compiler translates ActionScript ABC bytecode. That step is lossy: variable names become slot numbers, comments are gone forever, control flow is flattened into jump instructions.

Decompilation runs the arrow backwards: bytecode plausible source. It works because compilers are mechanical — their patterns are invertible. But it's lossy in the other direction:

- **Recovered:** exact logic, control flow, method names (if not obfuscated), string constants, class structure.
- **Lost forever:** comments, original variable names (with obfuscation), formatting, the author's "why".

Mnemonic: a decompiler is a microscope, not a time machine. You see the machinery in perfect detail; you don't recover the workshop.

Two views inside FFDec:

- **Decompiled (ActionScript 3)** — readable, high-level, inferred source. First resort.
- **P-code (raw bytecode)** — the actual instruction list. Use when the decompile output is broken or obfuscated; slower, but always ground truth.

Ch 6 — Tool: FFDec (JPEXS) in practice

FFDec = JPEXS Free Flash Decompiler — open source, Java, CLI + GUI. The workhorse of Flash archaeology.

```
java -jar ffdec-cli.jar -export script <outdir> shs3.swf
```

What the export found in SHS Duels:

- Every hardcoded URL — `http://www.strawhatsamuraiduels.com/server/` in `QueryManagerPIO.as`
- The response parsers: `QueryManagerBase.loginComplete()` reads login replies as **URL-encoded key=value** — not JSON! Fields: `status`, `chars`, `username`, `id`, `motd`, `magicnum`, `playertoken`, `playerrow`…
- The anti-piracy site-lock: `GameEngine.checkSiteLock()`
- Pieces of the PlayerIO multiplayer wiring — the parts we later chose NOT to fake (Ch 10).

The plot twist: the site lock accepts any URL containing `playerio`… or `strawhatsamuraiduels.com` or `explosivebarrel.net`. It's a **string search**, not a cryptographic check. The lock tells us the exact words our fake URL must contain — our replacement `http://playerio.localhost:8765/…` passes because we chose a name containing "playerio". **The anti-thief mechanism became our naming constraint.**

The wall we hit: FFDec's clean export worked only after the SWFProtection payload was decrypted (custom crackers — the deep end). For many games this step is unnecessary; for this one it was the price of admission.

Ch 7 — Reading the protocol out of the client

The quiet heart of the revival: reconstructing a dead server API with ZERO server access.

The client's parsing code IS the API documentation, one endpoint at a time. The chain:

1. Find what the client **sends**: URL + parameters (from QueryManagerPIO).
2. Find what the client **reads back**: the parser (e.g. `QueryManagerBase.loginComplete`).
3. Reproduce the exact server behaviour.

What loginComplete() taught us: the login response is NOT JSON — it's read like a query string, URL-encoded key=value pairs:

```
 status=0 & chars=… & username=Guest & id=1
motd=Welcome… & magicnum=0 & playertoken=… & playerrow=…
```

Any field the parser touches MUST appear, or the login screen breaks.

…And what the other endpoints expect (from the server we built):

- `saveplayer` body is the same urlencoded k=v form, sent BY the game; server answers `{"status": 0}` (JSON!)
- `getgold` `{"status": 0, "gold": p["playergold"]}`
- `getstats` `{"status": 0, "playerrow": …}`
- `gettown` / `getwar` `{"status": 0, "town": …, "war": {}}`

The protocol is a **POLYGLOT**: some endpoints speak form-encoding, others JSON. You only learn this by reading the parser. Never assume one format.

Method note: `playerrow` is a whole pre-formatted row string the client renders directly — some "fields" are mini-formats of their own. Data must be checked against the code that LOADS it, not only the file that stores it.

Ch 8 — Byte patching: the 43-character surgery

The SWF hardcodes its backend base URL. It doesn't ask DNS "where's the server?" — it contains the address as a literal string, like a fossil. The domain it points to is gone (NXDOMAIN) — so the client shouts into the void.

The equal-length invariant: the SWF is a binary container with length tables; strings are stored by size or referenced by offset. Replace a string with a different length everything shifts the file corrupts. The replacement must be byte-for-byte the same length:

```
 BEFORE (43 bytes): http://www.strawhatsamuraiduels.com/server/
AFTER  (43 bytes): http://playerio.localhost:8765/solo/server/
 same length — nothing after it shifts
```

"playerio" is kept in the replacement because `GameEngine.checkSiteLock()` only permits URLs containing playerio / strawhatsamuraiduels.com / explosivebarrel.net. Modern browsers resolve `*.localhost` themselves (RFC 6761) and macOS does it OS-wide; some OS resolvers (notably bare Windows setups) don't — if the game cannot connect, add the hosts line once (the README shows how).

The two patch sizes. The in-place string swap is the *smallest possible patch* — nothing else in the file moves (no recompile, no source, minimal blast radius). Use it when one string is all you need to change.

Our final build needed logic fixes too, so it went one size up: six classes edited and recompiled. **What the repo actually ships:** `shs3_solo.swf` = the unprotected decode (`shs3_decoded.swf`) + six recompiled classes — `QueryManagerPIO` (the 43-byte URL redirect + endpoint logic), `GameEngine` (offline boot, null guards, `SOLO:` diagnostics), `TitleUI`, `LoginForm`, `QueryManagerBase`, `StatsManager`. Measured: the file matches the decode for its first 3.61 MB and its trailing ~6 MB; only the ~1.5 MB band the recompiled classes occupy differs (the file is ~35 KB bigger overall). The audit trail is `patches/*.as` — readable diffs of everything changed.

Lesson: document the patch you actually made, not the smaller one you wish you'd made. A write-up claiming "we only swapped 43 bytes" while shipping a recompiled build would teach the wrong instinct — always state the patch surface honestly.

Ch 9 — Two kinds of emulators

Ruffle fakes the **machine**. Our server fakes the **service**.

	Platform emulator (Ruffle)	**Protocol emulator (solo_server.py)**
Fakes	the machine/runtime (Flash Player)	the service (dead PHP backend)
Layer	client-side, in the browser	server-side, on your laptop
Tech	Rust compiled to WebAssembly	Python, stdlib only (http.server)
Why needed	Flash Player is dead (2020); browsers refuse SWF	Backend is dead (NXDOMAIN); client needs answers

Ruffle re-implements Flash's behaviour so the SWF believes it runs in Flash Player. Bundled in our repo as index.html + ruffle.js + *.wasm — no install, just files next to the game.

solo_server.py does three jobs at once:

1. **Static server** — serves shs3_solo.swf, Ruffle, and mirrored assets (res/, data/, lang/)
2. **PHP emulator** — answers login, register, saveplayer, getgold, getstats, gettown, getwar… in the exact formats from Ch 7
3. **GameFS mirror** — routes `/r/<gameid>/<path>` to the local asset mirror, so even CDN calls resolve locally

State persistence: the fake backend saves character state to `.solo_state.json` — local saves, guest session. The game has no idea it's no longer talking to PlayerIO's database.

Ch 10 — Honest limits: what could NOT come back

A revival is a rescue operation, not a resurrection.

Permanently gone (as of Oct 2026):

- **True multiplayer duels** — PlayerIO ran authoritative game servers: matchmaking, live state sync, anti-cheat. Faking that means authoring a real-time game server — a much bigger project ("shs-solo" deliberately isn't one).
- **Real accounts / gold economy** — there is no "real" anymore; we fake a single Guest. The payments path is dead (whipflash 502) and we ignore it.
- **The developer's source code** — never needed! Decompiled logic + a faithful protocol was enough. That's the central lesson of this course.

Anti-overclaim: the README states the working/dead split in a table — solo overworld , saves , sound , PvP , payments . Claiming "the game is back" without that table would be a dishonest revival. Always document what works AND what cannot.

Ch 11 — The 7-step playbook (reusable) + glossary

Generalize one revival into a method:

1. **Prove what's dead.** DNS + HTTP probes on every URL the client references. Write a verdict per service.
2. **Acquire the front end.** Flashpoint archive, surviving CDNs, platform copies, web captures — two builds beat one.
3. **Decompile & read.** FFDec; read the decompiled view, drop to P-code when blocked; extract URLs + parsers + site-locks.
4. **Plan the smallest patch.** Find the site-lock's accepted strings first. Order of preference: one-string byte-swap targeted class-recompile full rebuild. Our revival needed the second (logic fixes), logged as readable `.as` patches.
5. **Write the emulator server.** One endpoint at a time, driven by the client's parsing code — never guess formats.
6. **Emulate the platform.** Ruffle or equivalent; bundle the emulator with the game so it "just works".
7. **Document honestly.** Working-vs-dead table, provenance and exact patches kept as readable artifacts.

Transfer beyond Flash: the same skeleton revives old Unity game servers, MMO private-server projects, dead mobile apps (their API clients still ship in the APK), abandoned desktop apps calling dead license servers. Swap the decompiler (FFDec Ghidra/IDA for native code), swap the platform emulator. The front-end-as-protocol-spec idea is universal.

Wayback census (2026-11-12) — definitive

Full archive map: `~/.jclaw/jclaw/workspace/main/shs-wayback-census-2026-11-12.md`.
Headlines:

- GameFS under our game ids: never archived. The local `res/`, `data/`, `lang/` mirror + decoded 2016-11-30 build are the ONLY known copies of all game data — this repo is a unique survival artifact (GitHub = second copy).
- The 2016 shell `shs3.swf`: single 2024 archive capture is encrypted junk (entropy ~8). Shell hunt closed.
- Kongregate game page: never archived; comments archaeology impossible via Wayback.
- Archived play.php (2014) proves the live loader embed host was r.playerio.com/r//shs3.swf.

LIVE DOOR DISCOVERED (2026-11-12 ~14:30 KL): explosivebarrel.net/duels/

The official site still serves the game TODAY: https://www.explosivebarrel.net/duels/ (live HTML, piwik-tracked) - buy it there; this copy is a local backup. It embeds `https://r.playerio.com/r/shsd-server1-eckroutikmljozkzitvg/shs3preloader.swf` + Ruffle (unpkg).

- NEW game id: **shsd-server1-eckroutikmljozkzitvg** (distinct from prod 5nsekx… and test no1ygg…).
- r.playerio.com file edge WORKS under this id (unlocked paths pulled 2026-11-12, sha256s in .shs-4a/):
  - shs3preloader.swf (164,533 B, magichtml-protected wrapper)
  - shs3_payload.swf (6,880,328 B) decrypted with the SAME key-104 recipe **payload-2026-DEC.swf 11,892,061 B** = the CURRENT server build; embeds shsd-server1 id; newgrounds(30)/facebook(19)/kong(6)/mindjolt(3) portals, no whipflash; lang: en/es/**fr** (NEW French)
  - res/shs3resource.swf 981,427 B (ours/prod-id: 623,535 — +58% content!), res/imageresource.swf 406,657 (ours 1,931,413 — smaller/split pack), bgresource/musicresource/soundresource identical to ours (byte-equal)
  - lang/shs3_en.xml 613,334 (ours 385,410 — +59% more strings), lang/shs3_es.xml 632,046, lang/shs3_fr.xml 637,132
  - LOCKED at the same policy door: data/*.xml (403), lang/shs3_zh-CN.xml (403), intro.mp4 (403), root paths (403)
- All files copied into repo: `payload-2026.swf`, `payload-2026-DEC.swf`, `preloader.swf`, `res-server1/`, `lang-server1/`.
- Politeness: 4a = 3 live GETs to explosivebarrel.net + forums (6s apart, old Safari UA, stop rule intact) + 8 r.playerio file-CDN reads (designed-for anonymous file access, code-referenced names only) + 0 knocks elsewhere.
- forums.explosivebarrel.com = parked lander (window.onload /lander). root explosivebarrel.net = live studio site (20,600 B).

2026-11-12 17:20 KL — CRATE FIX (decoded builds were structurally sick; now cured)

My Nov-11/Oct-30 decode sessions wrote the decrypted payloads as `CWS header + decompressed body` — a corrupt hybrid. Truth: the decrypted buffer IS already a proper CWS SWF (zlib body intact). Both decoded builds rebuilt verbatim from the encrypted originals:

- `payload-2026-DEC.swf` — proper CWS 6,880,328 B, sha256 21ba273456c0f4c3… (replaces commit 9a99e34's sick copy; sha256 of old: d76cd563c103249c…)
- (workspace) 2016-11-30 `shs3_payload_DECODED.swf` — proper CWS 6,880,715 B, sha256 3b13d0280ed44c23… First AVM2 boot of the 2026 build under Ruffle 0.7.1 (Chromium/Brave): boots, canvas 800×690, zero runtime errors; rendering stops after initial stage because the game's content loads via Loader (stub in ruffle) — harness continues in .shs-crate/.